

Python Conditionals and Loops

Nền Tảng Lập Trình Căn Bản

Tóm tắt nội dung

Câu lệnh điều kiện (conditionals) và vòng lặp (loops) là hai khái niệm cơ bản nhất trong lập trình. Chúng giúp máy tính đưa ra quyết định và lặp lại các tác vụ một cách thông minh. Bài giảng này sẽ hướng dẫn bạn từ những khái niệm đơn giản nhất, kết hợp với ví dụ minh họa và bài tập thực hành, để bạn có thể tự tin áp dụng vào các dự án thực tế.

Mục lục

1	Giới thiệu về Câu lệnh Điều kiện (Conditionals)	2
1.1	Câu lệnh if	2
1.2	Câu lệnh if-else	2
1.3	Câu lệnh if-elif-else	2
2	Giới thiệu về Vòng lặp (Loops)	3
2.1	Vòng lặp for	3
2.2	Vòng lặp while	3
3	Câu lệnh Điều kiện Nâng Cao	4
3.1	Toán tử logic	4
3.2	Câu lệnh lồng nhau (Nested Statements)	4
4	Vòng lặp Nâng Cao	5
4.1	Câu lệnh break và continue	5
4.2	Vòng lặp lồng nhau (Nested Loops)	5
5	Bài tập Thực hành	6
5.1	Bài tập Cơ bản	6
5.2	Bài tập Nâng cao	6
6	Kết luận	6

1. Giới thiệu về Câu lệnh Điều kiện (Conditionals)

Trong cuộc sống hàng ngày, chúng ta luôn đưa ra quyết định dựa trên điều kiện. Ví dụ: "Nếu trời mưa, tôi sẽ mang ô." Trong lập trình, câu lệnh điều kiện cho phép máy tính thực hiện các hành động khác nhau dựa trên điều kiện cụ thể.

Thông tin

Câu lệnh điều kiện trong Python giúp chương trình của bạn trở nên "thông minh" hơn bằng cách thực hiện các hành động khác nhau tùy thuộc vào tình huống.

1.1. Câu lệnh if

Cú pháp cơ bản của câu lệnh `if` trong Python:

```
if condition:
    # Code to execute if condition is True
```

Ví dụ

Ví dụ 1: Kiểm tra số dương

```
number = 5
if number > 0:
    print("This is a positive number.") # Output: This is a positive number.
```

Giải thích: Chương trình kiểm tra nếu `number` lớn hơn 0, thì in ra màn hình "This is a positive number."

1.2. Câu lệnh if-else

Khi bạn muốn thực hiện một hành động nếu điều kiện đúng, và một hành động khác nếu điều kiện sai, bạn sử dụng `if-else`:

```
if condition:
    # Code to execute if condition is True
else:
    # Code to execute if condition is False
```

Ví dụ

Ví dụ 2: Kiểm tra số chẵn/lẻ

```
number = 4
if number % 2 == 0:
    print("This is an even number.") # Output: This is an even number.
else:
    print("This is an odd number.")
```

Giải thích: Nếu `number` chia hết cho 2, chương trình in ra "This is an even number." Ngược lại, in ra "This is an odd number."

1.3. Câu lệnh if-elif-else

Khi có nhiều điều kiện cần kiểm tra, bạn sử dụng `if-elif-else`:

```
if condition1:
    # Code to execute if condition1 is True
elif condition2:
    # Code to execute if condition2 is True
else:
    # Code to execute if all conditions are False
```

Ví dụ

Ví dụ 3: Phân loại điểm số

```
score = 85
if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B") # Output: Grade: B
elif score >= 70:
    print("Grade: C")
else:
    print("Grade: F")
```

Giải thích: Chương trình kiểm tra điểm số và in ra xếp loại tương ứng.

2. Giới thiệu về Vòng lặp (Loops)

Vòng lặp cho phép bạn lặp lại một khối mã nhiều lần mà không cần viết đi viết lại. Python hỗ trợ hai loại vòng lặp chính: `for` và `while`.

2.1. Vòng lặp `for`

Vòng lặp `for` được sử dụng để lặp qua các phần tử trong một danh sách, chuỗi, hoặc một dãy số:

```
for item in sequence:
    # Code to execute for each item
```

Ví dụ

Ví dụ 4: In các phần tử trong danh sách

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
# Output:
# apple
# banana
# cherry
```

Giải thích: Vòng lặp `for` lặp qua từng phần tử trong danh sách `fruits` và in ra từng phần tử.

2.2. Vòng lặp `while`

Vòng lặp `while` lặp lại một khối mã miễn là điều kiện vẫn đúng:

```
while condition:
    # Code to execute while condition is True
```

Ví dụ

Ví dụ 5: Đếm ngược từ 5 đến 1

```
count = 5
while count > 0:
    print(count)
    count -= 1 # Output: 5, 4, 3, 2, 1
```

Giải thích: Vòng lặp while in giá trị của count và giảm count đi 1 cho đến khi count bằng 0.

3. Câu lệnh Điều kiện Nâng Cao

3.1. Toán tử logic

Python hỗ trợ các toán tử logic and, or, và not để kết hợp nhiều điều kiện:

- and: Trả về True nếu cả hai điều kiện đều đúng.
- or: Trả về True nếu ít nhất một điều kiện đúng.
- not: Đảo ngược giá trị của điều kiện.

Ví dụ

Ví dụ 6: Kết hợp điều kiện

```
age = 20
has_ticket = True
if age >= 18 and has_ticket:
    print("You can enter the event.") # Output: You can enter the event.
```

Giải thích: Chương trình kiểm tra nếu age lớn hơn hoặc bằng 18 và has_ticket là True, thì in ra "You can enter the event."

3.2. Câu lệnh lồng nhau (Nested Statements)

Bạn có thể lồng các câu lệnh điều kiện và vòng lặp vào nhau để giải quyết các bài toán phức tạp hơn.

Ví dụ

Ví dụ 7: Kiểm tra số nguyên tố

```
number = 7
is_prime = True
if number > 1:
    for i in range(2, number):
        if number % i == 0:
            is_prime = False
            break
if is_prime:
    print(f"{number} is a prime number.") # Output: 7 is a prime number.
else:
    print(f"{number} is not a prime number.")
```

Giải thích: Chương trình kiểm tra xem number có phải là số nguyên tố không bằng cách sử dụng vòng lặp for lồng trong câu lệnh if.

4. Vòng lặp Nâng Cao

4.1. Câu lệnh break và continue

- break: Thoát khỏi vòng lặp ngay lập tức.
- continue: Bỏ qua lần lặp hiện tại và tiếp tục với lần lặp tiếp theo.

Ví dụ

Ví dụ 8: Sử dụng break và continue

```
numbers = [1, 2, 3, 4, 5, 6]
for num in numbers:
    if num == 3:
        continue # Skip 3
    if num == 5:
        break # Stop loop at 5
    print(num)
# Output: 1, 2, 4
```

Giải thích: Vòng lặp bỏ qua số 3 và dừng lại khi gặp số 5.

4.2. Vòng lặp lồng nhau (Nested Loops)

Bạn có thể lồng các vòng lặp vào nhau để xử lý các cấu trúc dữ liệu phức tạp như ma trận.

Ví dụ

Ví dụ 9: In ma trận

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
for row in matrix:
    for element in row:
        print(element, end=" ")
    print()
# Output:
# 1 2 3
# 4 5 6
# 7 8 9
```

Giải thích: Vòng lặp ngoài lặp qua từng hàng của ma trận, vòng lặp trong lặp qua từng phần tử trong hàng.

5. Bài tập Thực hành

5.1. Bài tập Cơ bản

1. Viết chương trình kiểm tra một số có phải là số dương, âm, hay bằng 0.
2. Viết chương trình in ra các số chẵn từ 1 đến 10.
3. Viết chương trình tính tổng của các số từ 1 đến 100.

5.2. Bài tập Nâng cao

1. Viết chương trình kiểm tra một chuỗi có phải là chuỗi đối xứng (palindrome) hay không.
2. Viết chương trình in ra bảng cửu chương từ 1 đến 10.
3. Viết chương trình tìm số lớn nhất trong một danh sách.

6. Kết luận

Câu lệnh điều kiện và vòng lặp là nền tảng của lập trình. Chúng giúp bạn kiểm soát luồng thực thi của chương trình và tự động hóa các tác vụ lặp đi lặp lại. Hãy thực hành nhiều để thành thạo các khái niệm này!

Mẹo

Mẹo học tập:

- Thực hành viết code hàng ngày.
- Đọc và sửa lỗi trong các ví dụ mẫu.
- Áp dụng kiến thức vào các dự án nhỏ.